

Controllo del flusso

Condizioni in Python

Come insegnare al programma a prendere decisioni con if, elif ed else.

Perché le condizioni?

Un programma che fa sempre le stesse cose, senza mai adattarsi alla situazione, è poco utile. Le **condizioni** permettono al programma di prendere decisioni: fare una cosa se qualcosa è vero, un'altra cosa se è falso.

È come un semaforo: se la luce è verde, vai; se è rossa, fermati.

Il blocco **if** — "se"

La parola **if** significa "se" in inglese. Dice a Python: "esegui questo blocco di codice solo se la condizione è vera".

```
if condizione:
    # questo codice viene eseguito solo se la condizione è vera
```

Esempio pratico:

```
eta = 18

if eta >= 18:
    print("Sei maggiorenne.")
```

Se **eta** vale 18 o più, viene stampato il messaggio. Se vale meno di 18, non succede nulla.

:::tip[Ricorda l'indentazione!] Il codice dentro l' **if** deve essere rientrato di 4 spazi. Questa è la regola fondamentale di Python. :::

Il blocco `else` — "altrimenti"

`else` significa "altrimenti". Si usa per definire cosa fare quando la condizione dell' `if` è falsa:

```
eta = 15

if eta >= 18:
    print("Sei maggiorenne.")
else:
    print("Sei minorenne.")
```

Python controlla la condizione. Se è vera, esegue il blocco `if`. Se è falsa, esegue il blocco `else`. Non possono essere eseguiti entrambi.

Il blocco `elif` — "altrimenti se"

`elif` è l'abbreviazione di "else if" (altrimenti se). Serve quando hai più di due possibilità da gestire:

```
voto = 75

if voto >= 90:
    print("Ottimo!")
elif voto >= 70:
    print("Buono!")
elif voto >= 60:
    print("Sufficiente.")
else:
    print("Insufficiente.")
```

Python controlla le condizioni dall'alto verso il basso e si ferma alla **prima** che è vera. Le altre vengono saltate.

Con `voto = 75` :

1. `voto >= 90` ? No (75 non è >= 90)
2. `voto >= 70` ? Sì! → stampa "Buono!" e si ferma

Condizioni annidate

Puoi mettere un `if` dentro un altro `if`. Si chiama annidamento:

```
eta = 20
ha_patente = True

if eta >= 18:
    if ha_patente:
        print("Puoi guidare.")
    else:
        print("Sei maggiorenne ma non hai la patente.")
else:
    print("Sei minorenne, non puoi guidare.")
```

Ogni livello di annidamento aggiunge altri 4 spazi di indentazione. Tieni d'occhio gli spazi!

Combinare condizioni con `and` e `or`

Invece di annidare, spesso puoi combinare due condizioni in una sola riga:

```
eta = 20
ha_patente = True

# Con 'and': entrambe devono essere vere
if eta >= 18 and ha_patente:
    print("Puoi guidare.")

# Con 'or': basta che una sia vera
eta = 16
con_accompagnatore = True

if eta >= 18 or con_accompagnatore:
    print("Puoi entrare.")
```

Controllare se un valore è in una lista

L'operatore `in` è molto comodo per verificare se un elemento è presente in una sequenza:

```
frutta_disponibile = ["mela", "banana", "uva"]
scelta = "mela"

if scelta in frutta_disponibile:
    print("Ottima scelta, ce l'abbiamo!")
else:
    print("Mi dispiace, quella frutta non c'è.")
```

Esempio completo

```
numero = int(input("Inserisci un numero: "))

if numero > 0:
    print("Il numero è positivo.")
elif numero < 0:
    print("Il numero è negativo.")
else:
    print("Il numero è zero.")
```

Con questo programma:

- Se scrivi `5` → "Il numero è positivo."
- Se scrivi `-3` → "Il numero è negativo."
- Se scrivi `0` → "Il numero è zero."

Cicli for

Come ripetere un'azione per ogni elemento di una lista o sequenza.

Perché usare i cicli?

Immagina di dover salutare 100 studenti uno per uno. Senza i cicli, dovresti scrivere `print("Ciao!")` cento volte. Con un ciclo, lo scrivi una volta sola e Python lo ripete quante volte vuoi.

Il ciclo `for` in particolare è perfetto quando sai già **cosa** vuoi scorrere: una lista di nomi, le lettere di una parola, i numeri da 1 a 10.

Come funziona

```
for elemento in sequenza:  
    # codice da eseguire per ogni elemento
```

Ad ogni giro del ciclo, la variabile `elemento` assume il valore successivo della sequenza. Quando la sequenza finisce, il ciclo si ferma.

Scorrere una lista

L'uso più comune: fare qualcosa per ogni elemento di una lista.

```
frutta = ["mela", "banana", "ciliegia"]  
  
for frutto in frutta:  
    print(frutto)
```

Output:

```
mela  
banana  
ciliegia
```

La variabile `frutto` (che hai scelto tu) prende il valore `"mela"` al primo giro, `"banana"` al secondo, `"ciliegia"` al terzo.

Scorrere una stringa

Una stringa è una sequenza di caratteri, quindi puoi scorrerne le lettere una ad una:

```
for carattere in "Python":  
    print(carattere)
```

Output:

```
P  
y  
t  
h  
o  
n
```

Ripetere un numero preciso di volte con `range()`

Se vuoi eseguire qualcosa esattamente N volte, usa `range()` :

```
for i in range(5):  
    print("Ciao!")
```

Questo stampa "Ciao!" 5 volte. La variabile `i` vale 0, poi 1, poi 2, poi 3, poi 4.

Vedi la sezione su `range()` per tutti i dettagli su come usarla.

Sapere anche la posizione con `enumerate()`

A volte, mentre scorri una lista, vuoi sapere sia il valore sia la sua posizione (indice). Usa `enumerate()` :

```
frutta = ["mela", "banana", "ciliegia"]  
  
for indice, frutto in enumerate(frutta):  
    print(f"{indice}: {frutto}")
```

Output:

```
0: mela
1: banana
2: ciliegia
```

Puoi anche scegliere da quale numero partire:

```
for numero, frutto in enumerate(frutta, start=1):
    print(f"{numero}. {frutto}")
```

Output:

```
1. mela
2. banana
3. ciliegia
```

Scorrere due liste insieme con `zip()`

`zip()` abbina gli elementi di due liste, uno a uno:

```
nomi = ["Alice", "Bob", "Carlo"]
voti = [8, 7, 9]

for nome, voto in zip(nomi, voti):
    print(f"{nome} ha preso {voto}")
```

Output:

```
Alice ha preso 8
Bob ha preso 7
Carlo ha preso 9
```

Interrompere il ciclo: **break** e **continue**

break — ferma il ciclo immediatamente:

```
for i in range(10):
    if i == 5:
        break          # quando i vale 5, esci dal ciclo
    print(i)
# stampa: 0, 1, 2, 3, 4
```

continue — salta questo giro e va al prossimo:

```
for i in range(10):
    if i % 2 == 0:
        continue      # se il numero è pari, saltalo
    print(i)
# stampa solo i numeri dispari: 1, 3, 5, 7, 9
```

Cicli dentro cicli (annidati)

Puoi mettere un ciclo **for** dentro un altro. Per esempio, per costruire una tabellina:

```
for i in range(1, 4):
    for j in range(1, 4):
        print(f"{i} x {j} = {i * j}")
    print("----")
```

Output:


```
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
----
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
----
...
```

Scorrere un dizionario

I dizionari possono essere scorsi in modi diversi:

```
persona = {"nome": "Alice", "eta": 16, "citta": "Roma"}

# Solo le chiavi
for chiave in persona:
    print(chiave)    # nome, eta, citta

# Chiavi e valori insieme
for chiave, valore in persona.items():
    print(f"{chiave}: {valore}")
```

La funzione range()

Come generare sequenze di numeri per i cicli in Python.

Cos'è range() ?

`range()` è una funzione di Python che genera una **sequenza di numeri interi**. Si usa moltissimo nei cicli `for` quando vuoi ripetere qualcosa un numero preciso di volte, o quando vuoi lavorare con numeri in sequenza.

Pensa a `range()` come a un contatore automatico: tu gli dici da dove partire, dove fermarsi e di quanto saltare ad ogni passo.

Le tre forme di `range()`

Solo il numero finale: `range(fine)`

Genera numeri da `0` fino a `fine - 1` (il numero finale non è incluso):

```
for i in range(5):  
    print(i)
```

Output:

```
0  
1  
2  
3  
4
```

:::note[Perché parte da 0?] In programmazione si conta quasi sempre da 0. È una convenzione che incontrerai spesso.

`range(5)` genera 5 numeri, partendo da 0. :::

Inizio e fine: `range(inizio, fine)`

Genera numeri da `inizio` fino a `fine - 1`:

```
for i in range(2, 7):  
    print(i)
```

Output:

```
2  
3  
4  
5  
6
```

Il numero finale (7) non è incluso. È come dire "da 2 incluso a 7 escluso".

Con il passo: `range(inizio, fine, passo)`

Il terzo parametro indica di quanto avanzare ad ogni passo:

```
# Solo i numeri pari da 0 a 10
for i in range(0, 11, 2):
    print(i)
```

Output:

```
0
2
4
6
8
10
```

Il passo può anche essere negativo, per contare al contrario:

```
# Conto alla rovescia da 5 a 1
for i in range(5, 0, -1):
    print(i)

print("Via!")
```

Output:

```
5
4
3
2
1
Via!
```

Convertire range in una lista

Se vuoi vedere tutti i numeri generati da un `range`, convertilo in lista con `list()` :

```
print(list(range(5)))      # [0, 1, 2, 3, 4]
print(list(range(1, 6)))   # [1, 2, 3, 4, 5]
print(list(range(0, 10, 2))) # [0, 2, 4, 6, 8]
```

Esempi pratici

Sommare i numeri da 1 a 100

```
totale = 0
for i in range(1, 101): # da 1 a 100 incluso
    totale += i

print(totale) # 5050
```

La tabellina di un numero

```
n = 7
for i in range(1, 11):
    print(f"{n} x {i} = {n * i}")
```

Output:

```
7 x 1 = 7
7 x 2 = 14
...
7 x 10 = 70
```

Accedere agli elementi di una lista tramite posizione

```
frutta = ["mela", "banana", "ciliegia"]

for i in range(len(frutta)):
    print(f"Posizione {i}: {frutta[i]}")
```

Output:

```
Posizione 0: mela
Posizione 1: banana
Posizione 2: ciliegia
```

:::tip Nella maggior parte dei casi, è più semplice e leggibile usare `enumerate()` invece di `range(len(...))`. Vedi il capitolo sui cicli `for`. :::

Cicli while

Come ripetere un'azione finché una condizione rimane vera.

Il ciclo `while` — "finché"

Il ciclo `while` ripete un blocco di codice **finché una condizione rimane vera**. È diverso dal `for`: non sai in anticipo quante volte ripeterà, dipende da cosa succede durante l'esecuzione.

Pensa a un ascensore: le porte rimangono aperte finché non premi il pulsante di un piano. Non sai in anticipo quando lo premeranno.

```
while condizione:
    # codice da ripetere
```

Un primo esempio

```
contatore = 1

while contatore <= 5:
    print(contatore)
    contatore += 1 # aumenta il contatore di 1 ogni giro
```

Output:

```
1
2
3
4
5
```

Come funziona:

1. Python controlla: `contatore <= 5` ? Sì ($1 \leq 5$) → esegue il blocco
2. Stampa `1` , poi `contatore` diventa `2`
3. Controlla ancora: `2 <= 5` ? Sì → stampa `2` , `contatore` diventa `3`
4. ... e così via fino a che `contatore` diventa `6`
5. `6 <= 5` ? No → il ciclo si ferma

Attenzione al ciclo infinito!

Se la condizione non diventa mai falsa, il ciclo non finisce mai. Il programma si blocca per sempre:

```
# NON eseguire questo codice!
x = 0
while x < 10:
    print(x)
    # manca x += 1 - x non cambia mai, la condizione è sempre vera!
```

Se ti capita accidentalmente, puoi fermare il programma con **Ctrl+C** nel terminale.

Interrompere con **break**

break ferma il ciclo immediatamente, anche se la condizione è ancora vera. È utile quando vuoi uscire dal ciclo in base a qualcosa che succede dentro:

```
while True:          # questo ciclo non finirebbe mai da solo...
    risposta = input("Digita 'esci' per uscire: ")
    if risposta == "esci":
        break        # ...ma break lo ferma quando l'utente digita "esci"

print("Programma terminato.")
```

Saltare un giro con **continue**

continue salta il resto del codice in questo giro e passa subito al prossimo controllo della condizione:

```
contatore = 0

while contatore < 10:
    contatore += 1
    if contatore % 2 == 0: # se il numero è pari...
        continue        # ...salta la stampa e vai al giro successivo
    print(contatore)     # stampa solo i numeri dispari
```

Output:

```
1
3
5
7
9
```

Esempi pratici

Chiedere un input fino a quando è valido

Un uso classico del `while` : continuare a chiedere all'utente di inserire qualcosa finché non inserisce un valore accettabile.

```
while True:
    risposta = input("Inserisci 'sì' o 'no': ")
    if risposta == "sì" or risposta == "no":
        break
    print("Non ho capito, riprova.")

print(f"Hai scelto: {risposta}")
```

Conto alla rovescia

```
n = 5

while n > 0:
    print(n)
    n -= 1

print("Via!")
```

Output:

```
5
4
3
2
1
Via!
```


Quando usare `while` invece di `for` ?

- Usa `for` quando sai già **su cosa** vuoi iterare (una lista, un range, ecc.)
- Usa `while` quando vuoi ripetere qualcosa **finché una condizione cambia**, senza sapere in anticipo quante volte